GUÍA VISUAL PARA PRINCIPIANTES

Aprende a recorrer el sistema paso a paso

Una explicación sencilla para abrir el proyecto, entender sus partes y seguir una función desde la pantalla hasta la base de datos.

Proyecto Laravel · vistas Blade · PostgreSQL

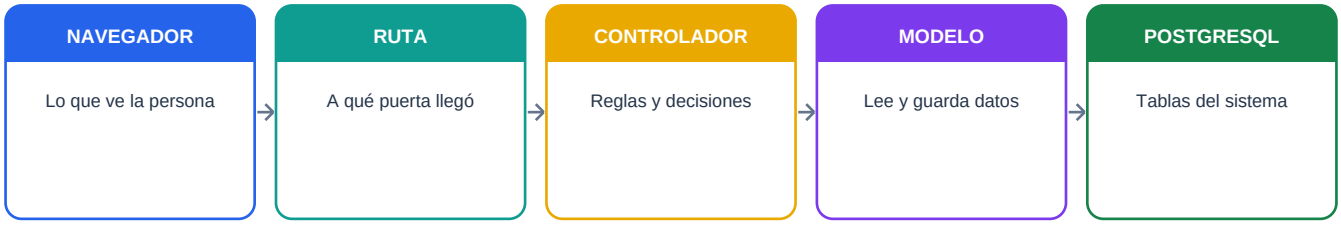
Preparada para estudiar el código en Visual Studio Code
20 de octubre de 2026

La idea principal
No necesitas aprender todo de golpe. Primero identifica qué archivo hace cada trabajo. Después sigue una solicitud por el camino: pantalla → ruta → controlador → modelo → PostgreSQL.

Este manual usa palabras comunes, ejemplos del proyecto y nombres de archivos reales. Los diagramas son mapas para estudiar, no pantallas nuevas del sistema.

1 · Primero, entiende la idea

Piensa en el sistema como una oficina que recibe una petición, revisa si está permitida, consulta un archivador y devuelve una respuesta.



El proyecto está unido en una sola aplicación Laravel. “Frontend” y “backend” son responsabilidades diferentes dentro del mismo proyecto; todavía no son dos aplicaciones separadas.

Palabras sin misterio

Palabra	En palabras sencillas	Un ejemplo aquí
Frontend	La parte que la persona ve y usa.	El calendario y sus botones.
Backend	La parte que recibe datos y aplica reglas.	Revisar permisos antes de guardar.
Base de datos	Un archivador ordenado en tablas.	La tabla `reservas`.
Servidor	El programa que responde al navegador.	Laravel recibe las solicitudes.

Recuerda
Un botón oculto no protege una acción. La autorización importante también debe revisarse en el backend.

2 · Abre el proyecto en Visual Studio Code

Antes de buscar archivos, abre la carpeta completa del sistema. Así las búsquedas y el terminal apuntan al proyecto correcto.

• En VS Code elige **Archivo** → **Abrir carpeta…** y selecciona la carpeta del proyecto, la que contiene el archivo `artisan`.

• En la barra izquierda, el icono de dos hojas abre el **Explorador**. Ahí puedes desplegar `app`, `routes`, `resources`, `database` y `tests`.

• Pulsa **Ctrl+P** para abrir un archivo por nombre; escribe, por ejemplo, `usar.blade.php`.

• Pulsa **Ctrl+Shift+F** para buscar texto en todo el proyecto; busca una ruta, un nombre de campo o un método.

• Abre **Terminal** → **Nueva terminal**. Comprueba que la ruta de la terminal termina en la carpeta del proyecto antes de ejecutar comandos.

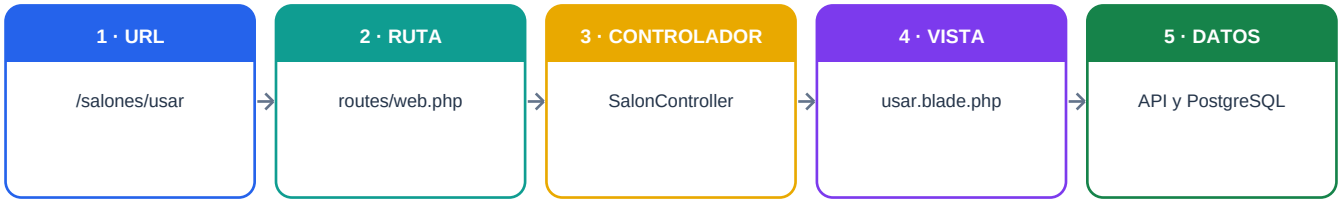
Mapa de carpetas que verás en VS Code

Carpeta	Qué suele guardar	Pregunta para orientarte
`routes/`	Direcciones del sitio.	¿Qué método atiende esta URL?
`app/Http/Controllers/`	Reglas para cada solicitud.	¿Qué revisa y qué devuelve?
`app/Models/`	Clases que acceden a datos.	¿Qué tabla representa?
`resources/views/`	Pantallas Blade.	¿Qué ve y rellena el usuario?
`database/migrations/`	Cambios de estructura de tablas.	¿Qué columnas crea el proyecto?
`tests/`	Comprobaciones automáticas.	¿Qué comportamiento se verifica?

No abras un archivo por accidente desde una copia
Asegúrate de trabajar en la carpeta del proyecto que contiene `artisan`, `composer.json` y `routes/`. Si VS Code muestra otra ruta, cierra la ventana y abre la carpeta correcta.

3 · Sigue una petición con los ojos

Ejemplo: alguien abre el calendario de salones. La pantalla no encuentra sola todas las reservas; el navegador pide esos datos al servidor.



Recorrido concreto

Paso	Archivo o pieza	Qué hace
1	`routes/web.php`	Conecta `/salones/usar` con la acción del calendario.
2	`SalonController::calendario()`	Prepara la vista y los datos de salones.
3	`resources/views/salones/usar.blade.php`	Dibuja la pantalla y ejecuta el JavaScript del calendario.
4	`/api/salones/reservas`	El navegador pide los eventos como datos JSON.
5	`ReservaController::obtenerReservas()`	Consulta reservas y salones y construye la respuesta.
6	FullCalendar	Presenta los eventos en la pantalla.

Un método es una acción con nombre
En `ReservaController::obtenerReservas()`, el archivo es el controlador y `obtenerReservas` es la acción que puedes buscar con Ctrl+Shift+F.

4 · ¿Qué parte cambio?

Busca primero el resultado que quieres. Si es apariencia, revisa la vista y sus estilos. Si es permiso o regla, revisa el backend. Una función completa puede tocar más de un archivo.

Si quieres cambiar…	Empieza revisando…	Por qué
Texto, botón o distribución	`resources/views/salones/usar.blade.php`	Allí está la plantilla de la pantalla.
Color global o modo oscuro	`resources/css/app.css`	Contiene estilos compartidos por la aplicación.
Una regla de reservas	`app/Http/Controllers/ReservaController.php`	Allí se validan y procesan acciones.
Quién puede eliminar	Controlador y `routes/web.php`	El permiso debe comprobarse en el servidor.
Una columna de base de datos	Modelo y nueva migración	El código y la estructura deben coincidir.
Una petición del navegador	Vista JavaScript + ruta + controlador	La pantalla envía datos al servidor.

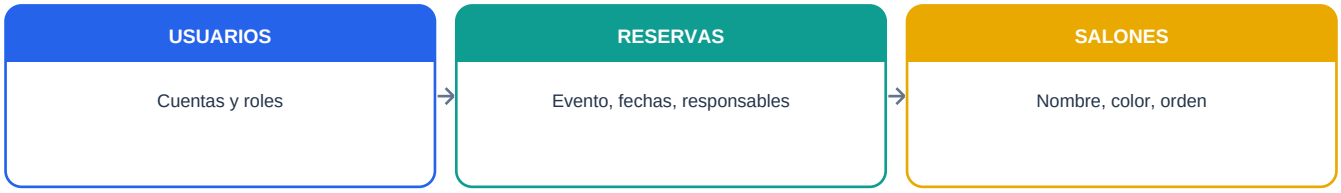
HTML, Blade, CSS y JavaScript

Tecnología	Qué significa aquí
HTML	Los campos, títulos y botones que estructuran la pantalla.
Blade	Plantilla de Laravel; `{{ }}` muestra datos y `@if` decide si mostrar contenido.
CSS	Reglas de aspecto: tamaño, color, separación y adaptación móvil.
JavaScript	Acciones en el navegador: abrir un calendario, validar una entrada o enviar una petición.
JSON	Formato de texto para que navegador y servidor intercambien datos.

Consejo de principiante
No intentes reorganizar una vista grande completa en tu primer cambio. Localiza una regla pequeña, entiende su función, cambia una cosa y observa el resultado.

5 · Entiende las tablas sin miedo

Una tabla se parece a una hoja de cálculo: columnas describen el tipo de dato y cada fila representa un registro. PostgreSQL es el motor de base de datos que usa este proyecto.



Leer un registro

Tabla	Columna	Ejemplo inventado	Qué representa
reservas	`id`	42	Número único del registro.
reservas	`titulo`	Visita guiada	Descripción corta del evento.
reservas	`fecha_inicio`	2026-10-22 08:30	Cuándo empieza.
reservas	`fecha_fin`	2026-10-22 09:30	Cuándo termina.
reservas	`salon`	Salón de Honor	Salón reservado; actualmente se guarda por nombre.
usuarios	`rol`	admin	Perfil que participa en las reglas de acceso.

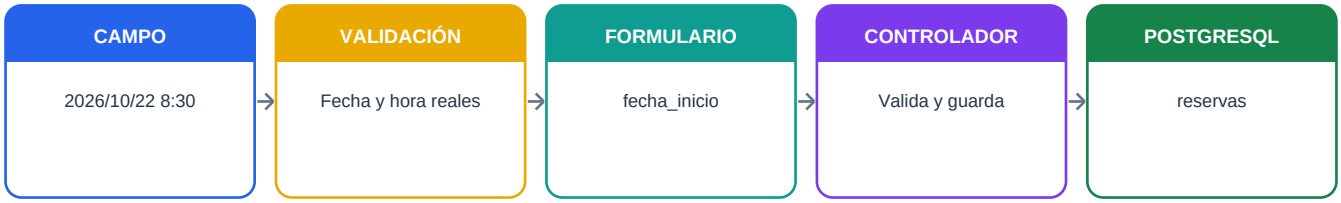
Qué es una clave

La clave primaria (PK) identifica una fila sin repetir, normalmente `id`. Una clave foránea (FK) es una conexión formal de una tabla a otra. En este proyecto, las migraciones no crean una FK entre `reservas.salon` y `salones.nombre`; la relación conceptual se hace por el nombre del salón. Eso significa que hay que comprobar que ese nombre siga coincidiendo.

Dónde comprobarlo en VS Code
Abre `database/migrations/` y busca `Schema::create`, `foreignId`, `unique` y los nombres de columnas. La migración es evidencia de lo que el proyecto quiere crear; compara con la base instalada antes de suponer que ambas coinciden.

6 · Cómo viaja una fecha desde el campo

Una fecha escrita y una elegida en el minicalendario terminan en el mismo valor para Laravel. La vista convierte el texto legible a un valor de fecha y hora para enviar el formulario.



Reglas para revisar

• El formato visible es `AAAA/MM/DD H:mm` o `AAAA/MM/DD HH:mm`; por ejemplo, `2026/10/22 8:30`.

• Las horas deben ir de 0 a 23 y los minutos de 00 a 59. El sistema rechaza fechas imposibles, como el día 30 de febrero.

• Al elegir otro día, el campo conserva la hora que ya tenía. Si no tenía hora, el calendario usa 08:30.

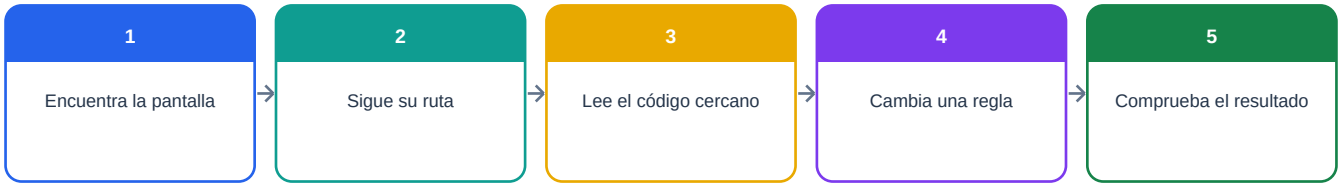
• El minicalendario compacto muestra días previos y siguientes para completar semanas; el número del mes ayuda a distinguirlos.

La vista que contiene los campos del calendario principal es `resources/views/salones/usar.blade.php`. La pantalla de supervisión tiene formularios equivalentes en `resources/views/salones/supervisar.blade.php`.

Prueba manual guiada
Abre una reserva de prueba, pulsa el campo para ver el minicalendario, cambia el mes, elige un día y comprueba que la hora no cambie. Después escribe una hora válida y envía solo si estás en un entorno de prueba.

7 · Haz cambios pequeños y comprensibles

Antes de editar, escribe en una frase qué comportamiento quieres modificar. Luego recorre los archivos en orden y no cambies varios temas a la vez.



Lista de comprobación para un cambio

Momento	Pregunta sencilla
Antes	¿En qué pantalla se nota el problema? ¿Qué archivo la dibuja?
Antes	¿La regla pertenece al servidor, al navegador o a ambos?
Durante	¿Puedo hacer una sola modificación y explicar por qué?
Después	¿La acción funciona con un dato correcto y uno incorrecto?
Después	¿Se ve bien en modo claro/oscuro y en una pantalla pequeña?
Guardar	¿El campo envía lo esperado y el servidor mantiene permisos?

Comentarios con fecha
Agrega fecha cuando registres una modificación concreta: `// 20/10/2026: acepta hora escrita con uno o dos dígitos.` Evita comentar cada línea; comenta el motivo que no es obvio.

Para distinguir partes, usa el estilo que corresponde: `// comentario` en JavaScript/PHP, `/* comentario */` en CSS y `{{-- comentario --}}` en Blade.

8 · Composer, npm y las librerías

Composer y npm son administradores de paquetes: instalan herramientas para el proyecto. No son lo mismo que el código propio del sistema.

Herramienta	Qué administra	Archivos para reconocer
Composer	PHP, Laravel y herramientas del backend.	`composer.json`, `composer.lock`, `vendor/`
npm	JavaScript, CSS y el proceso de compilación.	`package.json`, `package-lock.json`, `node_modules/`
Vite	Prepara recursos de frontend para el navegador.	`vite.config.js`, `resources/`, `public/build/`
Tailwind CSS	Clases utilitarias para dar estilo.	`tailwind.config.js`, archivos de vistas y CSS
FullCalendar	Dibuja el calendario de reservas.	Se carga desde CDN en la vista del calendario.

Comandos que podrías encontrar

Se escriben en el terminal de VS Code, desde la carpeta donde está `artisan`. Para ejecutar comandos en una base de datos, confirma primero que sea local o de pruebas.

```
composer install
npm install
php artisan serve
npm run dev
```

Qué es `vendor/` y `node_modules/`
Son carpetas generadas con paquetes de terceros. No corrijas un error del sistema editando allí; encuentra la configuración o el código propio del proyecto.

Versiones que aparecen en los archivos del proyecto: PHP 8.2+, Laravel 12, PostgreSQL, Vite 7, Tailwind CSS 3 y Alpine.js 3. Comprueba los archivos lock si necesitas saber la versión exacta instalada.

9 · Cuando algo no funciona

Un error es una pista. Anota qué hiciste, qué esperabas y qué ocurrió. Cambia una sola cosa después de localizar la primera pista útil.

Síntoma	Primer lugar para mirar	Qué buscar
La página dice 404	`routes/web.php`	¿Está registrada la URL?
La página dice 500	`storage/logs/laravel.log`	El error y la línea del controlador.
El calendario aparece vacío	Herramientas del navegador → Network/Red	Respuesta de `/api/salones/reservas`.
Un botón no responde	Herramientas del navegador → Console/Consola	Error JavaScript y nombre del archivo.
No se guarda la fecha	Campo, formulario y controlador	Formato, validación y respuesta de servidor.
Un cambio visual no aparece	CSS, Vite y caché del navegador	Si el archivo usado es el correcto y está guardado.

Ruta de revisión en el navegador

• Pulsa F12 para abrir las herramientas del navegador.

• En **Console**, lee el primer error rojo y copia el mensaje completo.

• En **Network**, recarga la página y abre la solicitud que falló. Revisa su código de respuesta.

• No compartas contraseñas, cookies ni tokens cuando compartas capturas o mensajes de error.

Errores de migración
Una migración cambia la estructura de la base. No uses `php artisan migrate:fresh` para “arreglar” un error: ese comando puede borrar tablas y datos. Primero confirma la conexión y el estado de las migraciones.

10 · Roles, permisos y datos seguros

Un rol describe qué acciones permite el servidor. Las reglas pueden variar por operación; no asumas que “administrador” significa acceso ilimitado a todo.

Pieza	Qué significa	Dónde estudiar
Autenticación	Confirma quién inició sesión.	`routes/auth.php`, controladores `Auth/`
Autorización	Decide si esa persona puede hacer una acción.	Métodos de controladores y middleware.
Validación	Comprueba que los datos recibidos tengan forma aceptable.	Controladores y `app/Http/Requests/`.
Contraseña	Debe guardarse protegida con hash, no como texto simple.	Modelo Usuario y flujo de autenticación.
CSRF	Protege formularios web contra solicitudes ajenas.	Middleware y formularios Laravel.

Hábitos que protegen datos

• No pegues claves o contraseñas en código, capturas, README o este documento.

• No publiques `.env`; usa `.env.example` para mostrar nombres de variables sin secretos.

• Antes de eliminar o cambiar datos, confirma el rol, el entorno y el efecto de la acción.

• En cambios de permisos, verifica el backend; una condición visual no sustituye la autorización.

Para aprender con seguridad
Usa una base de pruebas con datos ficticios para practicar. No ejecutes cambios destructivos en una base compartida o de producción.

11 · Plan de estudio de una semana

Dedica una sesión corta por tema. El objetivo es aprender a orientarte, no memorizar cada archivo.

Sesión	Qué leer	Ejercicio pequeño
1	`README.md` y esta guía.	Explica en voz alta qué es frontend y backend.
2	`routes/web.php`.	Encuentra la ruta `/salones/usar`.
3	`SalonController.php` y `ReservaController.php`.	Busca el método que carga el calendario.
4	`resources/views/salones/usar.blade.php`.	Encuentra el campo de fecha y el mini calendario.
5	`app/Models/Reserva.php` y migraciones.	Encuentra qué tabla y columnas usan las reservas.
6	`tests/` y `storage/logs/`.	Lee una prueba y localiza el formato del registro de error.
7	Repite el recorrido de una reserva.	Dibuja URL → ruta → controlador → base → vista.

Preguntas para comprobar que entendiste

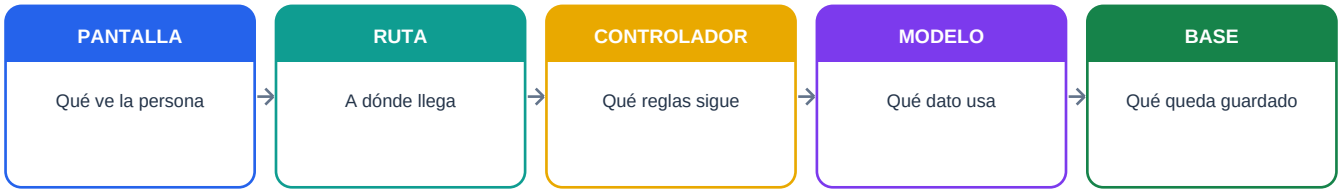
- ¿Qué archivo conecta una URL con un controlador?
- ¿Dónde debería verificarse si la persona puede eliminar una reserva?
- ¿Qué diferencia hay entre una vista Blade y un modelo?
- ¿Qué carpeta almacena la estructura deseada de la base de datos?

Respuestas rápidas
`routes/`; en el backend; la vista dibuja la pantalla y el modelo representa/consulta datos; `database/migrations/`.

12 · Glosario para tener a mano

Término	Definición fácil
Ruta	Dirección y método que conducen una solicitud al código adecuado.
Controlador	La parte que recibe, valida y decide qué respuesta devolver.
Modelo	La clase que representa datos y facilita consultar una tabla.
Migración	Instrucciones versionadas para crear o cambiar tablas y columnas.
Vista Blade	Plantilla que Laravel convierte en HTML para mostrar una página.
Endpoint	Una URL que recibe o devuelve datos; por ejemplo, una ruta API.
Dependencia	Una librería externa que el sistema utiliza.
Commit	Una fotografía guardada de cambios en Git.
Prueba	Una comprobación que ayuda a detectar si una conducta se rompe.

La ruta mental que conviene recordar



Referencia del proyecto
Esta guía complementa `docs/GUIA_DE_CODIGO_PARA_JUNIOR.md` y `README.md`. Revisión basada en el código disponible el 20/10/2026. Las configuraciones reales pueden variar por computadora y entorno.